

AI-901

Introduction to AI in Azure

Complete Course — Study Notes, Diagrams & Exam Preparation

Faculty Study Kit · Original content authored from public Microsoft Learn documentation
Exam weights: Identify AI concepts & capabilities (~40%) · Implement AI solutions with Microsoft
Foundry (~60%)

Contents

1. [Module 1 — AI Concepts & Getting Started with AI in Azure](#)
2. [Module 2 — Generative AI and Agents](#)
3. [Module 3 — Natural Language Processing & Text Analysis](#)
4. [Module 4 — AI Speech](#)
5. [Module 5 — Computer Vision](#)
6. [Module 6 — Information Extraction](#)

All diagrams and text in this document are original works created for this study kit. No Microsoft courseware material is reproduced. Official learning content: <https://aka.ms/mslearn-ai-901>

Module 1 — AI Concepts & Getting Started with AI in Azure

Exam mapping: *Identify AI concepts and capabilities* (~40%) and the foundations of *Implement AI solutions with Microsoft Foundry* (~60%). **Public references:** <https://aka.ms/mslearn-ai-concepts> · <https://aka.ms/mslearn-get-started-azure-ai>

1.1 What is Artificial Intelligence?

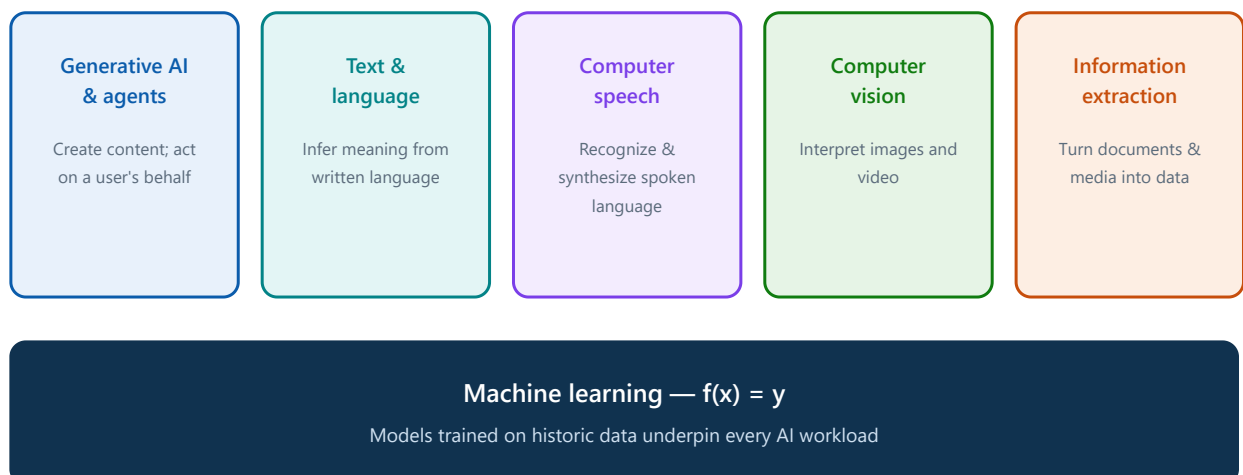
Artificial Intelligence (AI) is **software that imitates human capabilities**. In practice that means software able to:

- **predict outcomes and recognize patterns** from historic data,
- **evaluate content and make decisions**, then take an appropriate action,
- **understand and generate language**, holding natural conversations,
- **interpret visual input** such as images and video,
- **extract information** from documents and media to build knowledge.

AI is transforming business, education, healthcare, manufacturing, agriculture and more — but every capability above ultimately rests on **machine learning (ML)**: models trained on data to map an input to an output, conceptually $f(x) = y$.

1.2 The five common AI workloads

Common AI workloads

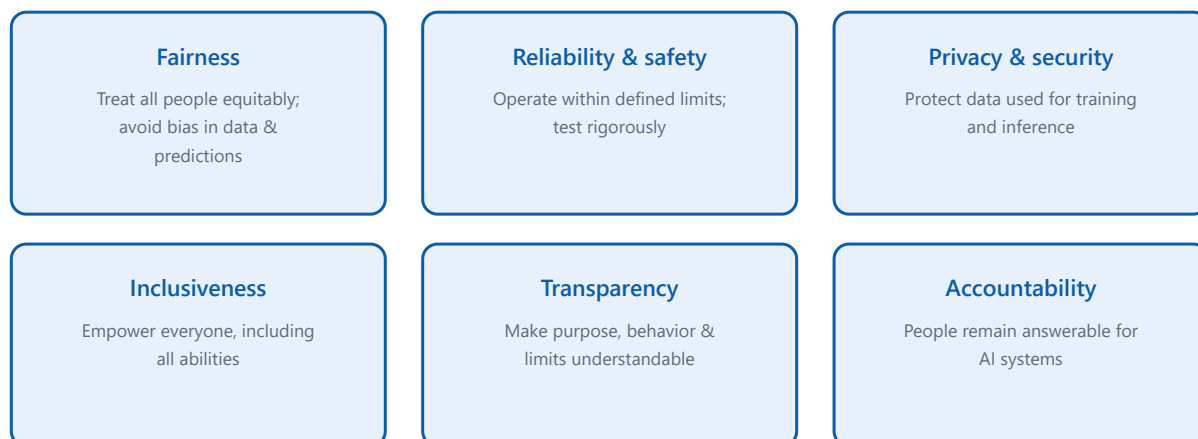


Workload	What it does	Typical examples
Generative AI & agents	Creates original content from prompts; agents act on a user's behalf	Chat assistants, copilots, automated workflows
Text & language (NLP)	Infers meaning from written language	Sentiment analysis, entity extraction, summarization
Computer speech	Recognizes and synthesizes spoken language	Transcription, voice assistants, screen readers
Computer vision	Interprets images and video	Object detection, image captioning
Information extraction	Converts documents/media into structured data	Invoice processing, call analysis

Key distinctions the exam loves: - *Generative AI* uses a **language model to create original content in response to a prompt** — it is **not** an "old" form of AI, and it is **not** restricted to data scientists. - An **AI agent** is *an application* that can perform tasks on behalf of a user — not a person.

1.3 Responsible AI — the six principles

Microsoft's six principles of responsible AI



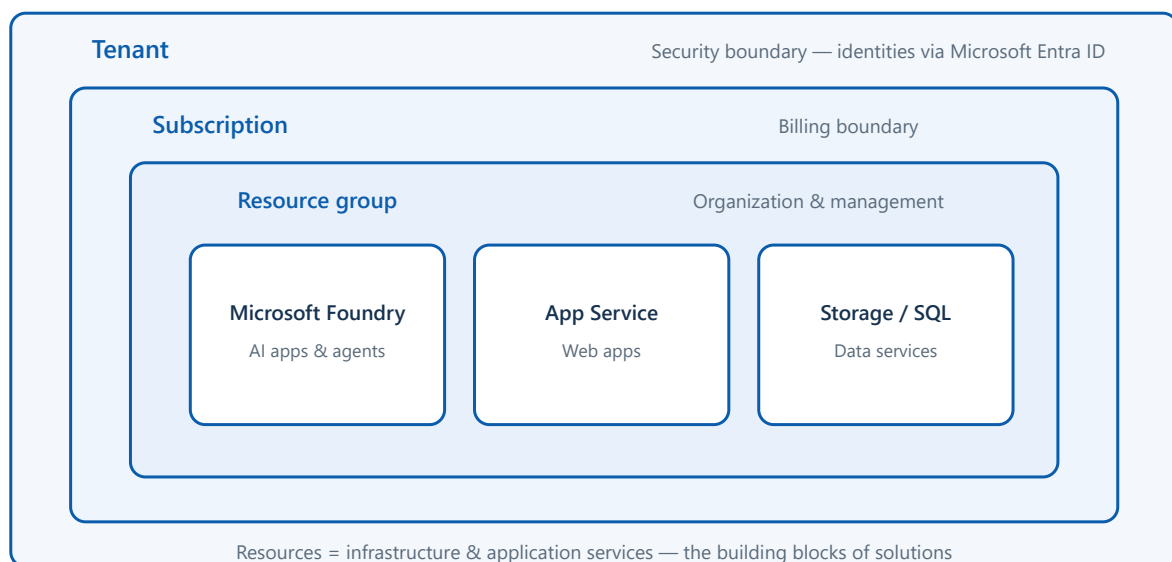
Principle	One-line meaning	Classic exam scenario
Fairness	Treat all people equitably; avoid bias	A loan model must not disadvantage a demographic group
Reliability & safety	Work correctly within defined limits; test rigorously	Autonomous vehicle / medical prediction must be validated

Principle	One-line meaning	Classic exam scenario
Privacy & security	Protect the data used to train and run models	Personal data must be secured during training & inference
Inclusiveness	Benefit everyone, including people with disabilities	Add captions/audio descriptions to reach all users
Transparency	Users should understand purpose, behavior, and limits	Publish what data a system uses and how it decides
Accountability	Humans stay answerable for AI decisions	Governance framework; a person can override the system

Tip: questions usually describe a scenario and ask *which principle applies*. Match the action word: bias→Fairness, testing→Reliability, personal data→Privacy, disability→Inclusiveness, "explain/understand"→Transparency, "who is responsible"→Accountability.

1.4 Microsoft Azure — where AI solutions live

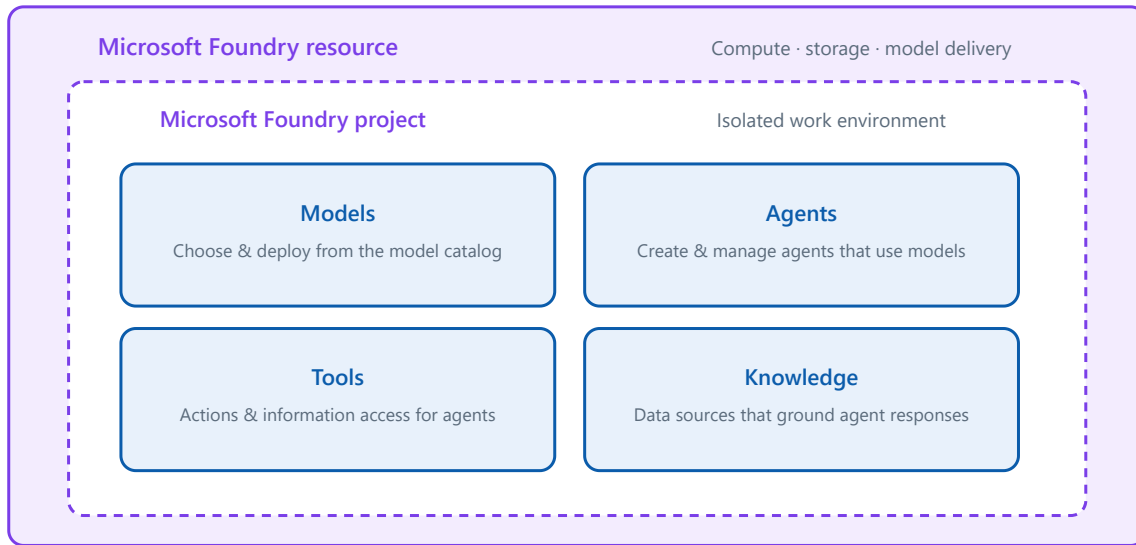
Microsoft Azure: organizational hierarchy



- A **tenant** is the organizational **security boundary**; identities (users, services) are managed and authenticated through **Microsoft Entra ID**.
- A **subscription** is the **billing boundary** inside a tenant.
- **Resource groups** organize and manage related resources.
- **Resources** are the building blocks — including **Microsoft Foundry**, Azure's platform for AI apps and agents.

1.5 Microsoft Foundry

Microsoft Foundry: resource, project, and assets

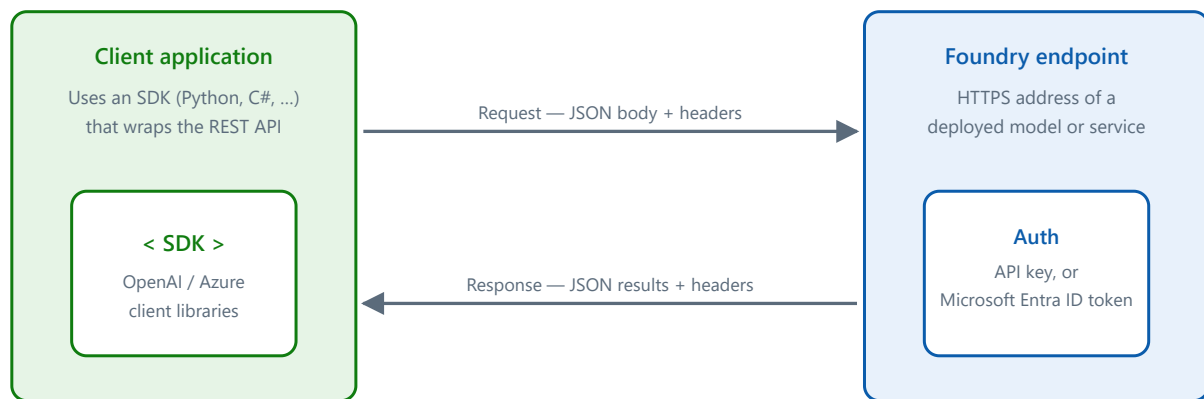


Microsoft Foundry is a unified platform for building AI apps and agents **on top of Azure** (it uses Azure compute, networking, identity and security — it does not replace Azure).

- A **Foundry resource** provides compute, storage, model delivery and related services.
- A **Foundry project** is an isolated work environment for development. Within a project you work with four asset types:
- **Models** — choose and deploy from an extensive multi-provider **model catalog**;
- **Agents** — create/manage agents that use models for language and reasoning;
- **Tools** — connect agents to capabilities so they can act and access information;
- **Knowledge** — manage the data sources agents use to ground their answers.

1.6 Endpoints, keys and SDKs

Foundry endpoints: how applications call models and services



Endpoint = where to call · Key/token = proof you may call it

- An **endpoint** is an **HTTPS address** an application calls to reach a deployed model or service.
- Requests are authenticated with an **API key** or a **Microsoft Entra ID token**.
- Endpoints expose **REST** interfaces: the request carries headers plus a **JSON** body; the response returns headers plus JSON results.
- In real projects you rarely call REST directly — **SDKs** (Python, C#, JavaScript, ...) wrap the API for you.

Memory hook: *endpoint = where to call; key = proof you're allowed to call it.* The key is never a storage location, and the endpoint never "stores secrets."

1.7 Quick self-check

1. Which boundary does a subscription define — security or billing? (*billing*)
2. Name the four asset types in a Foundry project. (*models, agents, tools, knowledge*)
3. A model must be tested to operate safely in bad weather — which principle? (*reliability & safety*)
4. What two things authenticate a call to a Foundry endpoint? (*API key or Entra ID token*)

Module 2 — Generative AI and Agents

Public references: <https://aka.ms/mslearn-intro-gen-ai> · <https://aka.ms/mslearn-get-started-gen-ai-agents>

2.1 What is generative AI?

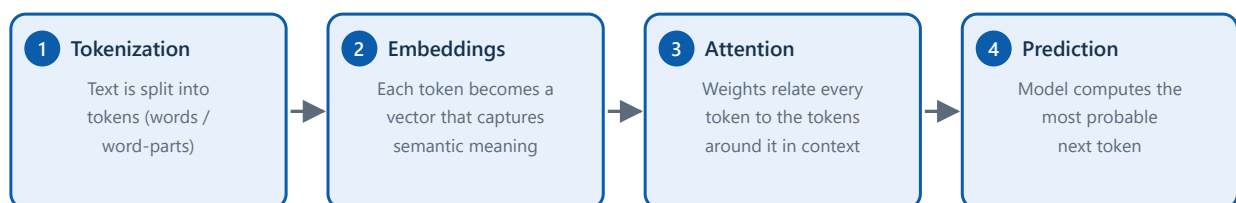
Generative AI produces **original content in response to a natural-language prompt**. Output can be natural language (text or speech), **images and video**, or **code** (Python, C#, SQL, ...).

Typical applications: - **smart chatbots** that generate answers dynamically instead of using hard-coded replies, - **creative assistants** producing original text, artwork and code, - the foundation of **agentic AI** — automation through smart, connected agents.

Generative AI is powered by **large language models (LLMs)**. **Small language models (SLMs)** are compact alternatives for devices and other resource-constrained environments.

2.2 How language models work

How language models work: tokens, embeddings, attention



Vector space intuition

Semantically similar tokens point in similar directions: "dog" $\approx$ "puppy", "cat" $\approx$ "kitten"

Vector arithmetic captures relationships: dog + young $\approx$ puppy

LLMs use the transformer architecture; SLMs are compact versions for constrained devices

Modern LLMs are built on the **transformer** architecture:

1. **Tokens** — text is split into small units (words or word-parts).
2. **Embeddings** — each token is encoded as a **vector** that captures semantic meaning.
Semantically similar tokens point in similar directions in vector space, and vector arithmetic

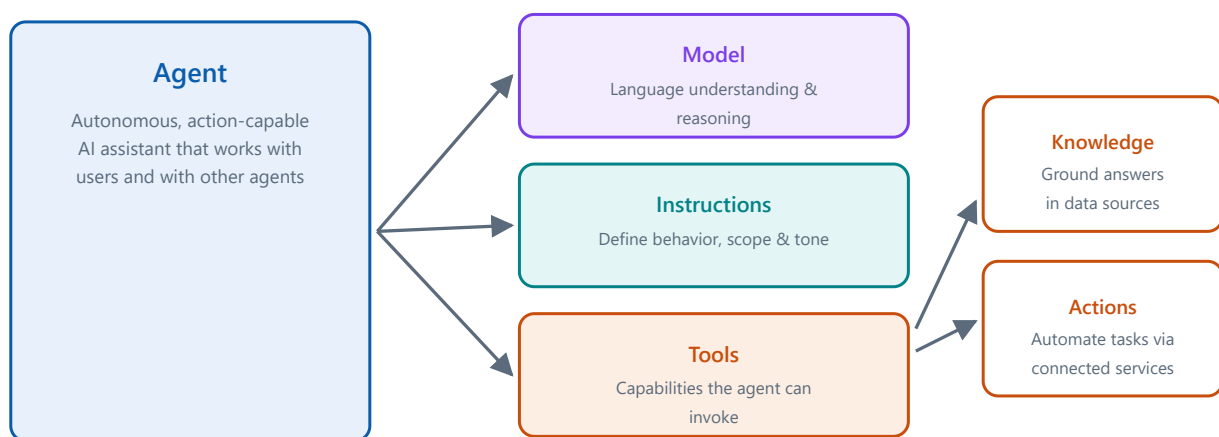
captures relationships (*dog + young ≈ puppy*).

3. **Attention** — for each token, the model weighs its relationship to the tokens around it. This is how context is encoded.
4. **Prediction** — using those attention-weighted vectors, the model repeatedly predicts the most probable *next token*, generating fluent output.

Exam anchors: *LLM* = AI model designed to generate human-like text. *Embeddings* = vector representations of tokens capturing semantic meaning. *Attention layer* = examines relationships between each token and the tokens around it.

2.3 What are agents?

Anatomy of an AI agent



An **agent** is an autonomous, action-capable AI assistant. It combines:

- a **model** — language understanding and reasoning,
- **instructions** — define behavior and scope (the "job description"),
- **tools** — used to find **knowledge** (grounding data) and perform **actions** (automation).

Agents collaborate with **human users** and with **other agents** — multi-agent workflows chain specialized agents, each with its own responsibility.

2.4 Foundry Models — the model catalog

The Foundry **model catalog** is a central hub to **discover, filter, compare and deploy** generative AI models from **multiple providers**:

- **Models sold directly by Microsoft** — billed through your Azure subscription (Azure OpenAI models and other collections).
- **Partner & community models** — purchased from trusted third-party providers.

Deployment considerations (know all four):

Setting	Controls
Deployment type	Where data is processed and how you pay
Version	Which model version + auto-update policy
Rate limits	Maximum tokens-per-minute (TPM) throughput
Guardrails	Responsible-AI content policies applied to the deployment

2.5 Using a generative model

Two ways to work with a deployed model:

1. **Model playground** (Foundry portal) — test prompts, compare models, tune settings, then grab working code. It does *not* remove the need for deployment or an API — it's for experimentation before you write code.
2. **OpenAI-compatible APIs / SDKs** — consume the model from application code.

You shape model behavior with three levers:

- **Instructions (system prompt)** — context and guidelines for *how* to respond.
- **Input (user prompt)** — the request itself; explicit, detailed prompts give better output.
- **Parameters** — e.g. **temperature** ("creativity"/randomness) and maximum response length.

```
from openai import OpenAI

client = OpenAI(base_url="<your-foundry-endpoint>", api_key="<your-api-key>")

response = client.responses.create(
    model="<your-model-deployment-name>",
    input="Explain the Turing Test in two sentences.")
print(response.output)
```

Reading the pattern: create a **client** bound to the endpoint + key → call `responses.create()` with the **deployment name** and the prompt → read the output.

2.6 Creating agents with the Foundry Agent Service

- Save a model configuration as a **named agent**, or build one directly.
- Add **tools**: *knowledge* (file search, web, data sources) and *actions* (connected services, code generation).
- Test in the **agent playground**.
- Connect from code with the **Foundry Project API**:

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project = AIProjectClient(endpoint="<project-endpoint>",
                          credential=DefaultAzureCredential())
agent = project.agents.get("expenses-agent")
openai_client = project.get_openai_client()

response = openai_client.responses.create(          # ← this line submits the prompt
    input=[{"role": "user", "content": "How do I submit an expense claim?"}],
    extra_body={"agent": {"name": agent.name, "type": "agent_reference"}})
print(response.output_text)
```

Exam anchor: the line that *submits a prompt to the agent* is the `responses.create(...)` call — `agents.get()` only fetches the agent definition and `get_openai_client()` only creates the client object.

2.7 Quick self-check

1. What architecture underpins modern LLMs? (*transformer*)
2. Which deployment setting caps tokens-per-minute? (*rate limits*)
3. Which parameter increases response randomness? (*temperature*)
4. Agent = model + ___ + ___. (*instructions + tools — knowledge & actions*)

Module 3 — Natural Language Processing & Text Analysis

Public references: <https://aka.ms/mslearn-nlp-concepts> · <https://aka.ms/mslearn-get-started-ai-text>

3.1 What is NLP?

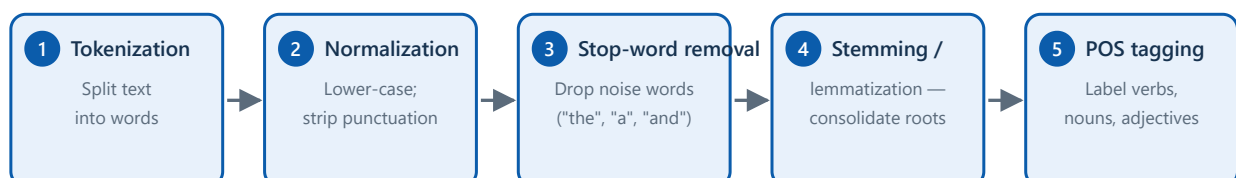
Natural language processing (NLP) is about **inferring meaning from text**. Core text-analysis tasks:

Task	Question it answers
Key-term / key-phrase extraction	What are the main topics?
Named entity recognition (NER)	Which people, places, organizations, dates appear?
Text classification (incl. sentiment analysis)	What category / tone is this?
Summarization	What's the short version?
Language detection	Which language is this written in?
PII detection & redaction	Which values are personally identifiable?

Common use cases: indexing documents for search, PII redaction, intent prediction for chatbots, spam filtering, social-media analysis, article categorization.

3.2 Text pre-processing

Text pre-processing pipeline



Example: "We choose to go to the Moon!" → choose (V) · go (V) · moon (N)

Prepared tokens feed statistical analysis or are vectorized for semantic models

Before analysis, raw text is prepared:

1. **Tokenization** — split text into words/units for analysis.
2. **Normalization** — remove capitalization and punctuation.
3. **Stop-word removal** — drop common "noise" words (*the, a, it, and*).
4. **Stemming / lemmatization** — consolidate word roots (*power, powered, powerful*).
5. **Parts-of-speech (POS) tagging** — label each word as verb, noun, adjective, ...

3.3 Statistical vs semantic analysis

Statistical techniques work on token counts:

- **Term Frequency (TF)** — most common tokens $\approx$ key topics of a document.
- **TF-IDF** — a word's frequency in *this* document **relative to** its frequency across *all* documents → importance of a word to a specific document within a collection.
- **Bag-of-Words (BoW)** — frequencies of words from a set correlate with a classification (e.g., *happy, great, fantastic* → positive).
- **TextRank** — sentence-relevance scoring (based on the PageRank idea), commonly used for **extractive summarization** (pick the n most relevant sentences).

Semantic models vectorize tokens as **embeddings** trained to capture meaning — this is the basis of modern LLMs. Embeddings capture semantic relationships in multiple dimensions; they are *not* translations or stop-word lists.

3.4 Text analysis in Microsoft Foundry — two routes

Text analysis in Foundry: choosing the right method

General-purpose AI models

(prompt an LLM, e.g. gpt-4.1-mini)

- ✓ Flexible, conversational analysis
 - ✓ Multiple tasks in one call
 - ✓ Natural-language responses
 - ✓ Exploratory workflows

X Probabilistic — results can vary
between identical calls

Azure Language (Foundry Tools)

(dedicated prebuilt service + SDK)

- ✓ Deterministic, structured output
- ✓ Confidence scores per result
- ✓ Production pipelines at scale
- ✓ PII detection for regulated data
- ✓ Language detection, NER, health

Strength: reproducible results

Same input → same output needed? Pick Azure Language. Flexibility needed? Prompt a model.

	General-purpose model (prompt an LLM)	Azure Language in Foundry Tools
Style	Flexible, conversational	Dedicated, prebuilt service
Output	Natural language; combine many tasks in one call	Deterministic, structured , with confidence scores
Best for	Exploration, ad-hoc analysis	Production pipelines, regulated data (PII), reproducible results at scale
Trade-off	Results may vary between calls (probabilistic generation)	Fixed task set

Azure Language capabilities include **PII extraction, language detection, named entity recognition, and Text Analytics for Health**.

Rule of thumb: *"same input must return the same structured result"* → **Azure Language SDK**.
"flexible natural-language analysis" → **general-purpose model via OpenAI API**.

3.5 Client applications

Route 1 — OpenAI library (general-purpose model): create an `OpenAI` client with endpoint + key, call `client.responses.create(model=deployment, input=prompt)`.

Route 2 — Azure Language SDK (`azure-ai-textanalytics`):

```
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential

client = TextAnalyticsClient(endpoint="<foundry-endpoint>",
                             credential=AzureKeyCredential("<key>"))

docs = ["Alex and I attended a Microsoft AI conference in Las Vegas on September 21st."]
result = client.recognize_entities(documents=docs)[0]
for e in result.entities:
    print(e.text, e.category, e.confidence_score)
```

- The **client object** is what lets your code communicate with the service (endpoint + key or Entra ID credentials).
- You submit a **collection of documents** for analysis.
- Handy methods: `detect_language()` → language name, ISO 639-1 code, confidence;
`recognize_pii_entities()` → **redacted text** + entity list + confidence per entity;

```
extract_key_phrases(), analyze_sentiment(), recognize_entities() .
```

3.6 Azure Language inside an agent (MCP)

Agents can call Azure Language through the **Model Context Protocol (MCP)**: the Azure Language **MCP server exposes Language capabilities to agents as tools**. You connect the tool to your agent (specifying the Foundry resource + credentials) and approve access **one time, always for this tool, or always for all tools**. The MCP server does not replace models — it gives agents a structured tool to call.

3.7 Quick self-check

1. Which technique weighs a term's document frequency against the whole corpus? (*TF-IDF*)
2. Which route gives deterministic output with confidence scores? (*Azure Language*)
3. What does `recognize_pii_entities()` return besides entities? (*redacted text + confidence scores*)
4. How do agents access Azure Language as a tool? (*via the MCP server*)

Module 4 — AI Speech

Public references: <https://aka.ms/mslearn-ai-speech-concepts> · <https://aka.ms/mslearn-get-started-ai-speech>

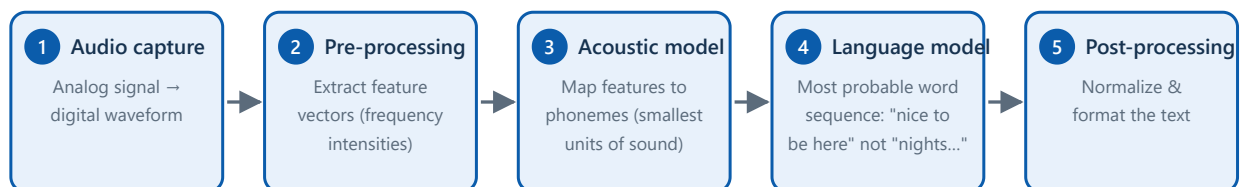
4.1 Speech-enabled solutions

	Speech recognition	Speech synthesis
Also called	Speech-to-text (STT)	Text-to-speech (TTS)
Input	Audio file, stream, or microphone	Text (file or user/app input)
Output	Text transcription	Audio waveform of spoken speech
Use cases	Customer support, voice assistants, automatic subtitles (accessibility), meeting transcription, clinical note-taking	Chatbots/conversational AI, screen readers (accessibility), notifications & alerts, e-learning, game character voices

Both directions appear in accessibility questions — captions = STT, screen reader = TTS.

4.2 How speech recognition works

Speech recognition (speech-to-text) pipeline



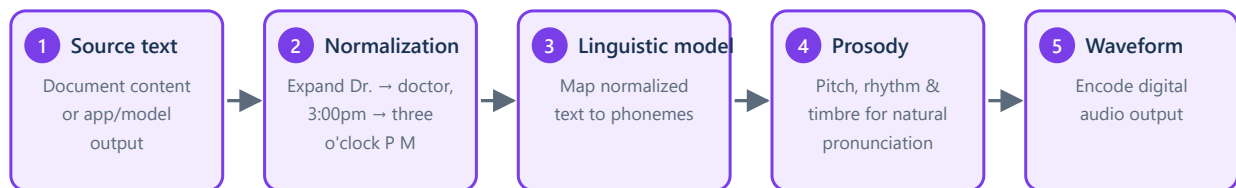
Output: transcribed text — used for captions, transcription, voice assistants

1. **Audio capture** — analog sound becomes a digital waveform.
2. **Pre-processing** — **feature vectors** are extracted from the waveform (signal intensity at specific frequencies). *Noise is removed, never added.*

3. **Acoustic modeling** — features are mapped to **phonemes**, the **smallest units of sound** in speech.
4. **Language modeling** — predicts the most probable **word sequence** from the phonemes (choosing "*nice to be here*" over "*nights two bee hear*").
5. **Post-processing** — normalize and format the final text.

4.3 How speech synthesis works

Speech synthesis (text-to-speech) pipeline



Output: spoken audio — used by assistants, screen readers, alerts, e-learning

1. **Source text** — document content or generated app/model output.
2. **Normalization** — expand abbreviations and numbers (*Dr.* → *doctor*, *3:00pm* → *three o'clock P.M.*).
3. **Linguistic modeling** — map normalized text to phonemes.
4. **Prosody generation** — adjust pitch, rhythm and timbre so pronunciation and cadence sound **natural** (prosody is not about volume or translation).
5. **Waveform encoding** — produce the digital audio output.

4.4 Speech in Microsoft Foundry

Azure Speech in Foundry Tools provides STT and TTS. Three ways to use it:

1. **Foundry portal playground** — experiment with real-time transcription (language identification, speaker diarization, output formats) and voice selection/tuning.
2. **Azure Speech SDK** — add speech directly to application code.
3. **MCP server** — expose speech capabilities to **agents** as tools.

Speech-to-text (Python SDK) — the pattern:

```
import azure.cognitiveservices.speech as speechsdk

speech_config = speechsdk.SpeechConfig(subscription="<key>", endpoint="<foundry-
endpoint>")
```

```
audio_config = speechsdk.audio.AudioConfig(filename="voice-message.wav")

recognizer = speechsdk.SpeechRecognizer(speech_config=speech_config,
                                         audio_config=audio_config)

result = recognizer.recognize_once_async().get()
print(result.text)
```

Text-to-speech — the pattern: configure `SpeechConfig` the same way, set `speech_synthesis_voice_name`, create a `SpeechSynthesizer`, call `speak_text_async(...).get()`.

No `AudioConfig` is needed for default speaker output (use one to write to a file instead). The SDK handles **authentication, network communication and audio generation** for you.

4.5 Voice Live — speech-capable agents

Azure Speech in Foundry Tools: Voice Live enables **real-time spoken conversation** with a generative AI model that has instructions and tools:

- continuous conversation flow with **interruption handling** and **background-noise reduction**;
- experiment in the Foundry portal playground;
- build client apps with the **Voice Live SDK (`azure-ai-voicelive`)** — it opens a real-time connection, streams audio, and handles spoken responses and interruptions.

4.6 Quick self-check

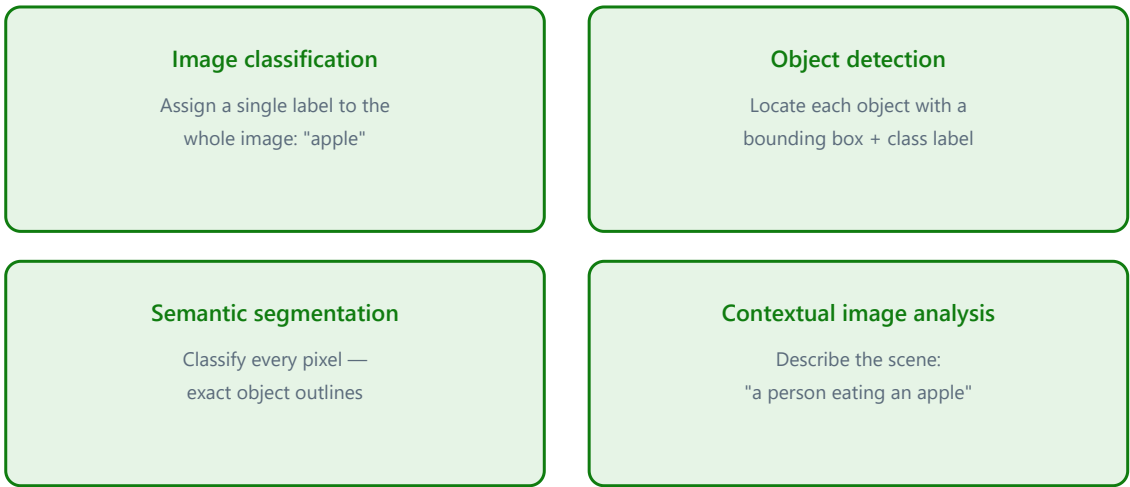
1. Phonemes are mapped by which model — acoustic or language? (*acoustic*)
2. What does prosody control? (*natural pronunciation & cadence — pitch, rhythm, timbre*)
3. In TTS code, when can you omit `AudioConfig` ? (*default speaker output*)
4. Which service powers real-time voice agents? (*Voice Live*)

Module 5 — Computer Vision

Public references: <https://aka.ms/mslearn-ai-vision-concepts> · <https://aka.ms/mslearn-get-started-ai-vision>

5.1 Vision tasks and techniques

Computer vision tasks



All manipulate and analyze pixel values, increasingly via multimodal models

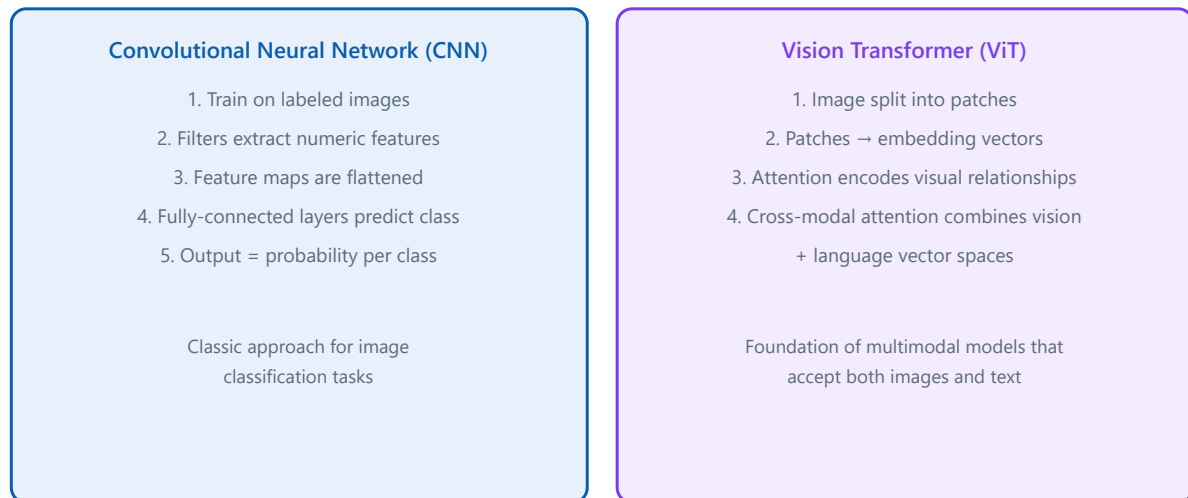
Computer vision is based on **manipulating and analyzing pixel values** (not file names or metadata). The core tasks, in increasing granularity:

Task	Output
Image classification	One label for the whole image ("apple")
Object detection	A bounding box + class label for <i>each</i> object
Semantic segmentation	A class for every pixel — exact object outlines
Contextual image analysis	A natural-language description of the scene (" <i>a person eating an apple</i> ")

Quick discriminator: "*locate each object*" → detection; "*pixel-level mask*" → segmentation; "*describe/caption the image*" → contextual analysis.

5.2 Vision models: CNN and ViT

Computer vision models: CNN vs Vision Transformer



Multimodal models let you prompt with an image + a question in a single request

Convolutional neural network (CNN) — the classic classifier: 1. train on images with known labels; 2. **filters extract numeric features** from the pixels (they are *not* cosmetic effects); 3. feature maps are flattened; 4. a fully connected network predicts the class; 5. output includes a **probability per class**.

Vision transformer (ViT) — the modern, transformer-based approach: 1. the image is split into **patches**, each turned into an embedding vector from pixel values; 2. **attention** encodes relationships between visual features in vector space; 3. **cross-modal attention** combines the visual vector space with language vectors — producing a **multimodal model** that understands images *and* text together.

A **multimodal model** can understand and work with **more than one type of data** (e.g. accept an image plus a text question in the same prompt).

5.3 Image analysis with multimodal models

Use the OpenAI API's **multi-part content** structure — one message containing an `input_text` part and an `input_image` part. Images can be **URLs or base64-encoded data**:

```
import base64
from openai import OpenAI

client = OpenAI(base_url="<foundry-endpoint>", api_key="<api-key>")

with open("banana.png", "rb") as f:
```

```

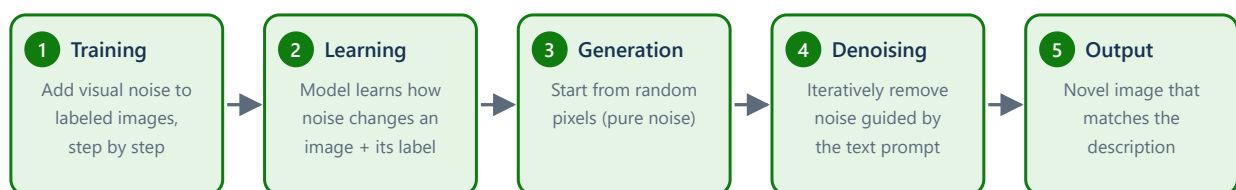
image = base64.b64encode(f.read()).decode("utf-8")

response = client.responses.create(
    model="<model-deployment-name>",
    input=[{"role": "user", "content": [
        {"type": "input_text", "text": "What desserts can I make with this?"},
        {"type": "input_image", "image_url": f"data:image/png;base64,{image}"}]]])
print(response.output[0].content[0].text)

```

5.4 Generating images and video

Diffusion models: how images are generated



Video generation (e.g. sora models) runs as an asynchronous job — poll until complete

Diffusion models power image generation: - **Training:** visual "noise" is added to labeled images step by step; the model learns how noise changes an image with a given description. - **Generation:** the process is reversed — start from **random pixels** and iteratively remove the learned noise guided by the text prompt, revealing a novel image.

In Foundry: - **Image generation** — search the catalog for "**text to image**" inference-task models (e.g. OpenAI **gpt-image** family); generate programmatically by sending text prompts through the **OpenAI Responses API** against your deployed image model. - **Video generation** — search for "**video generation**" models (e.g. OpenAI **sora**). Video jobs run **asynchronously** because generation is **resource-intensive and takes time**: you submit the job, then poll for completion.

5.5 Quick self-check

1. Which task labels every pixel? (*semantic segmentation*)
2. What do CNN filters do? (*extract numeric features from images*)
3. Two accepted image formats in a multimodal prompt? (*URL or base64 data*)
4. Why is sora video generation asynchronous? (*resource-intensive, takes time*)

Module 6 — Information Extraction

Public references: <https://aka.ms/mslearn-ai-info-concepts> · <https://aka.ms/mslearn-get-started-information-extraction>

6.1 What is information extraction?

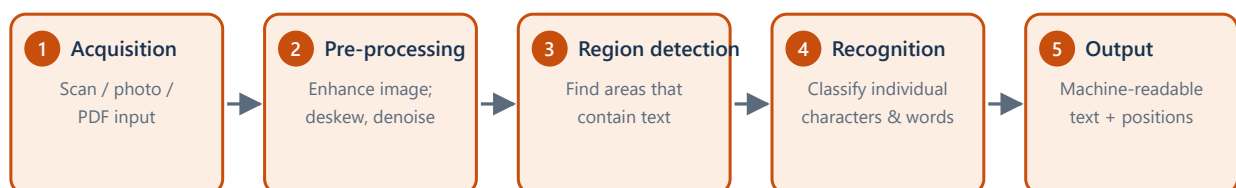
Information extraction means **analyzing unstructured content to identify and extract relevant fields and values** — converting digital content into useful, structured data. Sources include:

- documents and emails,
- business cards and receipts,
- invoices, contracts and other forms,
- images,
- audio recordings and video.

Classic scenario: a photographed receipt becomes a pre-filled expense claim (vendor, date, amount) with no manual typing.

6.2 Optical Character Recognition (OCR)

Optical Character Recognition (OCR) pipeline



OCR reads the text; field extraction gives that text business meaning

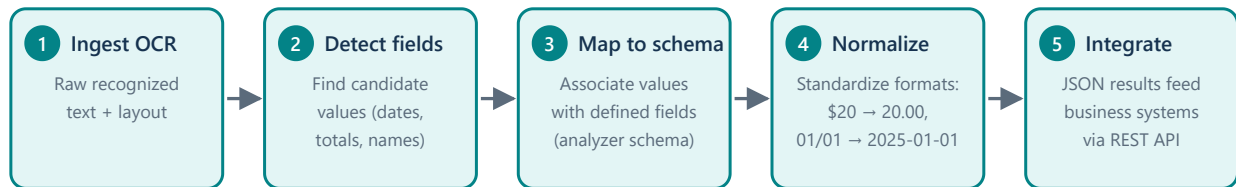
OCR converts images of text into machine-readable text data. The pipeline:

1. **Image acquisition & input** — scan, photo or PDF.
2. **Pre-processing & enhancement** — deskew, denoise, sharpen.
3. **Text region detection** — find the areas that contain text.
4. **Character recognition & classification** — identify the characters and words.
5. **Output generation & post-processing** — machine-readable text with positions.

OCR *reads* the text — it doesn't know what the text *means*.

6.3 Field extraction and mapping

Field extraction & mapping (Azure Content Understanding)



Predefined analyzers for invoices/receipts; custom analyzers from your own sample schema

Turning OCR output into business data:

1. **OCR output ingestion** — raw recognized text + layout.
2. **Field detection & candidate identification** — spot values that look like dates, totals, names.
3. **Field mapping & association** — attach each value to the right schema field. Generative AI improves this step by **using semantic language models to match extracted values to data fields accurately** (no hand-coded rules per document type).
4. **Data normalization & standardization** — \$20 → 20.00 , 01/01/2025 → 2025-01-01 .
5. **Integration** — structured results flow into business processes and systems.

6.4 Azure Content Understanding in Foundry Tools

Azure Content Understanding is Foundry's information-extraction service. Its key advantage over plain OCR: it **understands document structure and maps extracted data to a defined schema** (with confidence scores per field).

- **Analyzers** define **how content is processed and what structured data is returned**:
- **predefined analyzers** for common documents — invoices, receipts, contracts, call recordings;
- **custom analyzers** — define your own schema from **sample documents**.
- Results are retrieved as **JSON via the REST API**.
- With the Python SDK, analysis is **asynchronous**: you submit content, then **poll a URL until the analysis job completes** (results are not returned in the same request).

6.5 Audio and video extraction

The same analyzer concept applies to media:

- **post-call analysis** (contact centers — sentiment, topics, speakers),
- **voice-message automation**,
- **video call transcription and summary**,
- **video recording analysis**.

6.6 Quick self-check

1. What does OCR stand for and do? (*optical character recognition — images of text → machine-readable text*)
2. What defines the schema and processing behavior in Content Understanding? (*an analyzer*)
3. After submitting content via the SDK, how do you get results? (*poll until the job completes*)
4. Content Understanding vs OCR in one line? (*CU adds structure + schema mapping on top of reading text*)